

# Contents

|                                                                                                                        |    |
|------------------------------------------------------------------------------------------------------------------------|----|
| 1. How does the ledger system prevent race conditions and "double-spending" during concurrent transactions? . . . . .  | 3  |
| 2. What deadlock prevention strategy is implemented during fund transfers between accounts? .                          | 4  |
| 3. How does the system handle transaction idempotency for payment gateway top-ups and webhooks? . . . . .              | 5  |
| 4. Why does the application use stateful HTTP sessions instead of stateless JWTs for user authentication? . . . . .    | 5  |
| 5. How is legacy password hash migration handled without disrupting the user experience? . . .                         | 6  |
| 6. What rate-limiting architecture is employed to defend against brute-force attacks on sensitive endpoints? . . . . . | 7  |
| 7. How are dynamic QR payments secured against tampering and replay attacks? . . . . .                                 | 8  |
| 8. What is the purpose of the security filter bypassing CSRF validation for Stripe webhooks? . .                       | 9  |
| 9. How is the database connection pool optimized to prevent application-level startup failures? .                      | 10 |

10. What are the performance and concurrency implications of synchronous file writing in the logging utility? . . . . . 11
11. What security vulnerability exists in the OTP generation mechanism, and how should it be remediated? . . . . . 12
12. How does the database schema ensure non-repudiation and automated security auditing? . . 13
13. What CPU-bound denial of service risk is present in the loan calculation logic? . . . . . 14
14. What are the memory risks associated with generating PDF statements in-memory? . . . . . 15
15. How does the application prevent XSS attacks at the input validation and utility levels? . . . 16
16. Why does the application use database constraints like CHECK constraints for transaction types and statuses? . . . . . 17
17. How does the application handle Supabase or cloud database connections securely? . . . . . 18
18. What transaction management issues arise from having duplicate transaction logic in servlets and services? . . . . . 19
19. How are email communications protected against SMTP configuration leaks? . . . . . 20

|                                                                                                                                   |    |
|-----------------------------------------------------------------------------------------------------------------------------------|----|
| 20. What failure handling mechanism is implemented during user signup to prevent orphaned database records? . . . . .             | 21 |
| 21. How are admin dashboards and system settings protected from unauthorized access? . . . . .                                    | 22 |
| 22. What is the impact of utilizing <code>initializationFailTimeout = -1</code> on system monitoring and health checks? . . . . . | 23 |
| 23. Why are indexes placed on timestamps and active flags in the banking database schema? . .                                     | 24 |
| 24. What security risks are associated with Stripe webhook processing if the webhook secret is missing? . . . . .                 | 24 |
| 25. How does the application handle timezone differences for transaction dates and OTP expirations?                               | 25 |

## 1. How does the ledger system prevent race conditions and “double-spending” during concurrent transactions?

Answer:

- Utilizes explicit row-level locking via `SELECT balance FROM accounts WHERE account_id =`

? FOR UPDATE within database transactions.

- Introduces database locks that block subsequent reads/updates on the same account until the current transaction commits or rolls back.
- Increases connection hold times and latency but guarantees mathematical correctness and transaction serialization.

## **2. What deadlock prevention strategy is implemented during fund transfers between accounts?**

Answer:

- Consistently orders the acquisition of row locks by comparing account IDs, always locking the account with the smaller ID first.
- Eliminates circular wait conditions that occur when two threads concurrently attempt to transfer funds between the same pair of accounts in opposite directions.
- Requires small computation overhead before lock acquisition to prevent database-level deadlocks.

### **3. How does the system handle transaction idempotency for payment gateway top-ups and webhooks?**

Answer:

- Checks if a transaction with a unique reference number already exists in the transactions table before executing any deposit logic.
- Maps the globally unique Stripe Event ID as the reference number (STRIPE\_ + eventId) in the ledger database.
- Prevents duplicate credit operations during network retries or duplicate webhook delivery from Stripe.

### **4. Why does the application use stateful HTTP sessions instead of stateless JWTs for user authentication?**

Answer:

- Enables immediate session revocation from the database/session manager, which is a critical regulatory compliance requirement for compromised banking accounts.
- Avoids security risks associated with JWT storage in client-side localStorage, which is highly vulnerable to Cross-Site Scripting (XSS).
- Requires server-side memory to store session state, limiting horizontal scalability without sticky sessions or external caches like Redis.

## **5. How is legacy password hash migration handled without disrupting the user experience?**

Answer:

- Implements a dual-verification check in the authentication utility that validates inputs against both BCrypt and legacy SHA-256 hashes.
- Automatically re-hashes verified SHA-256 passwords using BCrypt (12 rounds) during successful logins and updates the database record.

- Prolongs the login request latency slightly for legacy users but secures user records seamlessly over time without password reset flows.

## **6. What rate-limiting architecture is employed to defend against brute-force attacks on sensitive endpoints?**

Answer:

- Employs an in-memory ConcurrentHashMap within a custom filter that tracks POST requests to login and OTP endpoints.
- Groups requests by client IP, parsing X-Forwarded-For and X-Real-IP headers to identify users behind load balancers.
- Limits memory footprint by using short TTL windows, though it does not synchronize state across multiple server nodes in a clustered environment.

## 7. How are dynamic QR payments secured against tampering and replay attacks?

Answer:

- Signs the payment payload (receiver account, amount, and millisecond timestamp) using an HMAC-SHA256 signature generated with a server secret.
- Enforces a strict 15-minute expiration window by verifying the payload timestamp against the current server time during payment execution.
- Prevents malicious users from modifying transfer amounts or reusing generated QR codes, though it relies on synchronized server clocks.

## **8. What is the purpose of the security filter bypassing CSRF validation for Stripe webhooks?**

Answer:

- Selectively excludes the Stripe webhook path from the general CSRF filter as webhook requests originate from external Stripe servers.
- Delegates security validation for these endpoints to cryptographic webhook signature verification using the Stripe SDK.
- Ensures webhook processing remains functional while protecting all other state-changing browser requests from cross-site request forgery.

## 9. How is the database connection pool optimized to prevent application-level startup failures?

Answer:

- Configures HikariCP with `initializationFailTimeout = -1` to allow the servlet container to start successfully even if the database is unreachable.
- Prevents hard crashes during application initialization and allows the connection pool to attempt database reconnects lazily.
- Delays database connectivity errors until the first query execution instead of failing fast during deployment.

## 10. What are the performance and concurrency implications of synchronous file writing in the logging utility?

Answer:

- Writes log entries directly to disk using synchronous `FileWriter` instances in the thread executing the HTTP request.
- Blocks servlet threads during high disk I/O, introducing transaction processing delays and thread starvation under heavy load.
- Guarantees immediate persistence of critical security and error logs, but must be migrated to asynchronous appenders in production.

## **11. What security vulnerability exists in the OTP generation mechanism, and how should it be remediated?**

Answer:

- Uses `java.util.Random` to generate 6-digit OTP codes, which uses a predictable linear congruential formula.
- Allows attackers to predict future OTP values if they reconstruct the seed, compromising signups, password resets, and profile updates.
- Must be replaced with `java.security.SecureRandom` to ensure cryptographically strong, non-deterministic number generation.

## 12. How does the database schema ensure non-repudiation and automated security auditing?

Answer:

- Implements PostgreSQL triggers (user\_update\_audit, transaction\_create\_audit) that execute auditing functions on update and insert statements.
- Automatically logs password changes, profile status updates, and successful transactions directly into audit tables in real-time.
- Protects the audit trail from application-level tampering, although it increases write latency on the database server.

## 13. What CPU-bound denial of service risk is present in the loan calculation logic?

Answer:

- Computes monthly interest using `BigDecimal.pow(tenure)` where `tenure` is parsed directly from user input without range validation.
- Allows malicious actors to submit excessively high tenure numbers (e.g., millions of months), exhausting CPU cycles or triggering `OutOfMemoryErrors`.
- Requires strict request validation to cap loan tenures to reasonable values before executing exponentiation operations.

## **14. What are the memory risks associated with generating PDF statements in-memory?**

Answer:

- Loads transaction lists and renders A4 landscape PDF documents entirely within JVM heap memory using `ByteArrayOutputStream`.
- Risk of out-of-memory errors (OOM) if multiple concurrent users request large full statements spanning thousands of transactions.
- Requires pagination of transaction queries and streaming PDF outputs directly to the servlet response output stream for larger reports.

## 15. How does the application prevent XSS attacks at the input validation and utility levels?

Answer:

- Sanitizes input parameters using a utility method that replaces standard HTML characters with their safe character entities.
- Implements `SecurityHeadersFilter` to set response headers that block browser-based scripting exploits.
- Lacks contextual HTML escaping, making it vulnerable to bypasses if input is rendered in non-standard HTML attributes or script tags.

## **16. Why does the application use database constraints like CHECK constraints for transaction types and statuses?**

Answer:

- Enforces data integrity at the database storage engine layer, blocking invalid transactions regardless of the client application state.
- Prevents database corruption from developer bugs or direct SQL edits that bypass application logic validations.
- Requires database schema modifications to add new transaction types or statuses, limiting dynamic configuration flexibility.

## 17. How does the application handle Supabase or cloud database connections securely?

Answer:

- Configures the connection pool with `sslmode=require` as a datasource property to enforce TLS encryption in transit.
- Prevents eavesdropping and man-in-the-middle attacks when transmitting sensitive banking data over public networks.
- Introduces minor cryptographic handshake overhead for database connections, which is mitigated by connection pooling.

## **18. What transaction management issues arise from having duplicate transaction logic in servlets and services?**

Answer:

- Features inline JDBC transactions in `TransactionServlet` that bypass the structured lock ordering and validation found in `LedgerService`.
- Increases code maintenance overhead and creates security vulnerabilities where race conditions could occur.
- Requires refactoring all financial updates to route through the centralized, deadlock-safe `LedgerService`.

## **19. How are email communications protected against SMTP configuration leaks?**

Answer:

- Loads all SMTP settings (host, port, credentials) strictly from environment variables using a zero-hardcoding secret policy.
- Masks email addresses in application logs (revealing only the first two characters and the domain) to protect customer privacy.
- Relies on manual environment configurations, which can cause mail delivery failures if credentials expire.

## **20. What failure handling mechanism is implemented during user signup to prevent orphaned database records?**

Answer:

- Bundles the insertion of the user record and the creation of their primary savings account inside a single database transaction.
- Rolls back the entire transaction if either the user insertion or account creation throws an exception, keeping the database consistent.
- Increases the execution time of the signup process but prevents orphaned, account-less user profiles in the database.

## 21. How are admin dashboards and system settings protected from unauthorized access?

Answer:

- Implements `AdminAuthFilter` which intercepts all requests to the `/admin/*` path and verifies session credentials.
- Records unauthorized admin access attempts and login failures inside the database security logs with IP addresses and user agents.
- Lacks role-based access control (RBAC), treating all authenticated admins with uniform privileges across the dashboard.

## 22. What is the impact of utilizing `initializationFailTimeout = -1` on system monitoring and health checks?

Answer:

- Prevents container health checks from reporting app failure during deployments if the cloud database is starting up slowly.
- Allows the application container to boot and serve static error pages or retry connections gracefully.
- Hides database connection failures during startup, requiring active application probes to monitor true database availability.

## **23. Why are indexes placed on timestamps and active flags in the banking database schema?**

Answer:

- Optimizes query performance for statement generation, ledger exports, and expired OTP cleanups.
- Reduces query execution time for critical database operations like verifying pending OTPs and loading transaction histories.
- Enhances read latency but introduces minor write-amplification during insert and update operations.

## **24. What security risks are associated with Stripe webhook processing if the webhook secret is missing?**

Answer:

- Decodes Stripe event payloads directly without signature verification if the webhook secret is empty

or unset.

- Permits attackers to spoof payment completed webhooks by sending mock JSON payloads to the endpoint.
- Must be secured in production by throwing a servlet initialization error if the webhook secret is missing.

## **25. How does the application handle timezone differences for transaction dates and OTP expirations?**

Answer:

- Records transactions using PostgreSQL's default `TIMESTAMP` (which defaults to database server local time) and verifies OTP expirations in JVM local time.
- Risk of out-of-sync checks or incorrect transaction reporting if the application server and database server run in different time zones.
- Requires migrating timezone-sensitive columns to `TIMESTAMP WITH TIME ZONE (TIMESTAMPTZ)`

and using UTC timestamps throughout.